

1. ¿Qué es la clase Conexión y cuál es su papel?

La clase Conexion es una clase interna (internal) que centraliza toda la comunicación con la base de datos MySQL. Su papel es evitar repetir código de conexión en cada formulario, agrupando en un solo lugar la apertura/cierre de la conexión y todas las consultas del proyecto (totales, tops, informes, etc.).

2. ¿Por qué se creó el método conectar() y qué función cumple?

Se creó para inicializar y abrir la conexión usando la cadena url, que contiene el servidor, base de datos, usuario, puerto y contraseña. Se llama al inicio de cada método público antes de ejecutar cualquier consulta, y si falla muestra un MessageBox con el error.

3. ¿A qué hace referencia la clase Conexión y qué elementos administra?

Administra tres elementos clave: el objeto conn (de tipo MySqlConnection), la cadena de conexión url con los parámetros del servidor remoto (195.35.53.72), y todos los métodos de consulta que obtienen datos para los formularios, como obtenerTotales, CargarGridTop3Tienda, CargarGridInforme1, etc.

4. ¿Qué tipo de clase es MySqlConnection y cuál es su propósito?

Es la clase del conector oficial de MySQL para .NET (MySQL.Data.MySqlClient). En el código se instancia como conn = new MySqlConnection(url) y representa la conexión física con el servidor. Sin ella, ninguna consulta puede ejecutarse.

5. ¿Qué es MySqlCommand y para qué se utiliza?

Es la clase que permite enviar instrucciones SQL al servidor. En el proyecto se usa en obtenerTotales: MySqlCommand comando = new MySqlCommand(consulta, conn), seguido de comando.ExecuteReader() para leer los tres contadores globales (libros, volúmenes y ventas).

6. ¿Qué relación existe entre MySqlCommand y MySqlConnection?

MySqlCommand depende de MySqlConnection para saber a qué servidor enviar la consulta. Al escribir `new MySqlCommand(consulta, conn)` se le pasa tanto la instrucción SQL como la conexión activa `conn`, de modo que el comando sabe exactamente dónde ejecutarse.

7. ¿Por qué se crean clases en POO y qué ventajas ofrece?

En este proyecto, tener la clase `Conexion` separada del resto significa que los formularios no necesitan saber nada sobre MySQL; simplemente llaman a métodos como `CargarGridInforme1(dgv)`. Esto facilita el mantenimiento: si cambia el servidor o la contraseña, solo se modifica la variable `url` en un único lugar.

8. ¿Qué son los métodos dentro de una clase y qué función cumplen?

Son las acciones que ejecuta la clase. Aquí cada método tiene una responsabilidad concreta: `conectar()` abre la conexión, `cerrarConexion()` la cierra, y métodos como `CargarGridInforme3(dgv, piso)` ejecutan una consulta específica y devuelven o muestran sus resultados. Todos son `static` porque no se necesita instanciar la clase.

9. ¿Qué es MySqlDataAdapter y qué papel cumple?

Actúa de intermediario entre la consulta SQL y un `DataTable`. En el proyecto se usa en casi todos los informes: `MySqlDataAdapter da = new MySqlDataAdapter(consulta, conn)` seguido de `da.Fill(dt)`, lo que ejecuta la consulta y vuelca todos los resultados directamente en la tabla en memoria para luego asignarla al `DataGridView`.

10. ¿Qué es DataTable y por qué algunos métodos pueden ser estáticos?

`DataTable` es una representación en memoria de los resultados de una consulta, con filas y columnas. En el proyecto se usa como paso intermedio: se llena con el `DataAdapter` y se asigna como `dgv.DataSource = dt`. Los métodos de la clase son estáticos porque no dependen del estado de ningún objeto concreto de `Conexion`, sino que operan directamente sobre la conexión compartida `conn` y los parámetros recibidos.